

Contents

1	Types of variables	2
1.1	Numerical (quantitative)	2
1.2	Categorical (qualitative)	2
2	More plots (ggplot2 continued)	3
3	Bar/column plots (geom_bar()/geom_col())	3
4	Boxplots (geom_boxplot())	4
4.1	Zooming in on a plot (coord_cartesian())	4
5	Histograms (geom_histogram()) and density plots (geom_density())	6
5.1	Histograms (geom_histogram())	6
5.2	Density plots (geom_density())	7
6	Matrix scatter plots	8
6.1	The iris dataset	8
6.2	Scatter plots for clustering (geom_point() with color)	9
6.3	Matrix scatter plots with the pairs() function	9
6.4	Matrix scatter plots with the psych package (pairs.panels())	10

1 Types of variables

Broadly, there are two types of variables that we will be concerned with for now: **numerical** variables and **categorical** variables. There are other types that we may encounter in future tutorials.

1.1 Numerical (quantitative)

Numerical variables (also referred to as quantitative variables in the tutorial sheet) are, as the name suggests, numerical, and may be *continuous* or *discrete*. For example, the height and weight of someone is a (continuous) numerical variable. In the `flights` dataset, the air time and distance are examples of numerical variables. Typically, these variables **can be compared** to another variable of the same type. (e.g. you can compare weights, heights, etc. to be greater than, less than, equal to another)

```
flights <- nycflights13::flights
head(flights$air_time)
```

```
## [1] 227 227 160 183 116 150
```

There are some variables which **may appear to be numerical** but are not. For example, the flight number in the `flights` dataset (`flights$flight`) consists only of numbers, but there is no logical way to compare a flight number to another.

```
head(flights$flight)
cat("The flight number is an", typeof(flights$flight), end="\n")
cat("There are", length(unique(flights$flight)), "different flight
↪ numbers")
```

```
## [1] 1545 1714 1141 725 461 1696
## The flight number is an integer
## There are 3844 different flight numbers
```

1.2 Categorical (qualitative)

Categorical variables, as a simplified description, are variables that are not numerical. In previous tutorials, we have grouped the `flights` dataset by `carrier`, which is a categorical variable (with 16 different carriers). The flight number is also a categorical variable.

```
unique(flights$carrier)
cat(length(unique(flights$carrier)), "unique carriers")
```

```
## [1] "UA" "AA" "B6" "DL" "EV" "MQ" "US" "WN" "VX" "FL" "AS" "9E" "F9" "HA" "Y"
## [16] "OO"
## 16 unique carriers
```

2 More plots (ggplot2 continued)

Last week, we learnt about scatterplots and boxplots briefly. We will see them again this week, along with a few new kinds of plots.

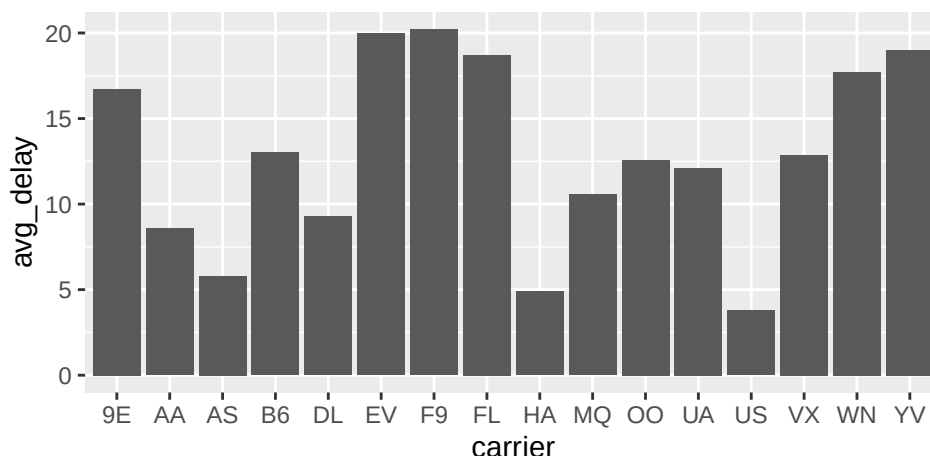
3 Bar/column plots (geom_bar()/geom_col())

Bar/column plots are used to visualise the *difference in values* (or lack thereof), especially with a *smaller number of observations*. For example, you could visualise the GDP per capita or population of the 10 ASEAN nations with a bar/column plot. The example in the tutorial sheet uses it on **summary statistics** of each carrier's average delay (16 entries).

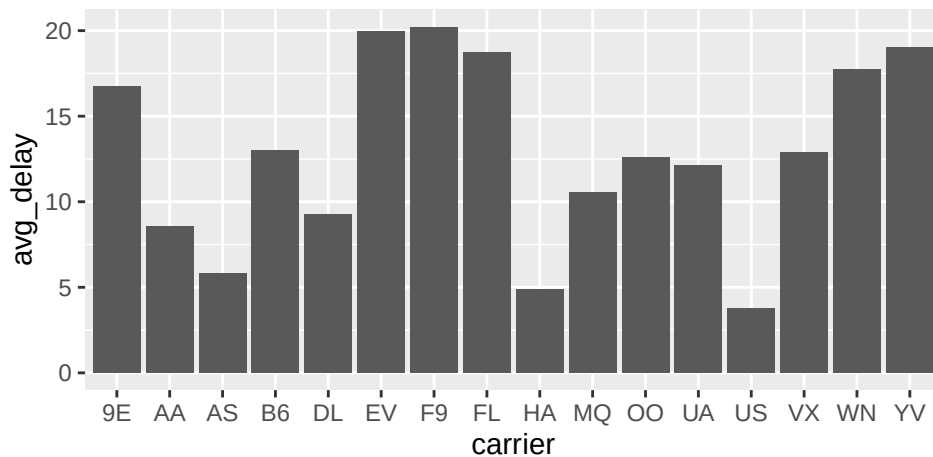
Typically, bar plots are used to visualise the **count** of observations. If you look at the documentation of `geom_bar()`, the default value for the `stat` parameter is `'count'`. **This means that typically, `geom_bar()` is used to count the number of observations of a variable in a dataset.**

The notes change this `stat` parameter to `identity` so that `geom_bar()` plots the values that it is passed, i.e. the average departure delays computed. Alternatively, `geom_col()` could be used without changing any parameters.

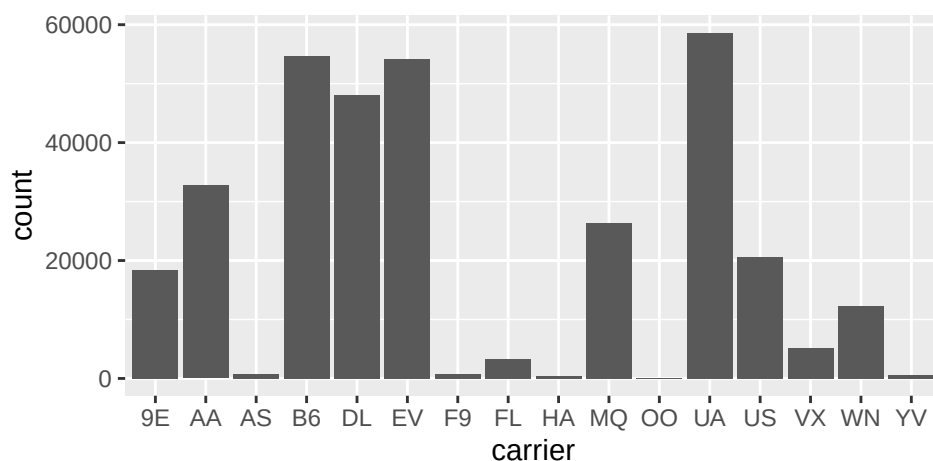
```
df <- flights %>% group_by(carrier) %>%  
  summarise(avg_delay = mean(dep_delay, na.rm=T))  
  
ggplot(df) + geom_bar(aes(carrier, avg_delay), stat = 'identity')
```



```
ggplot(df) + geom_col(aes(carrier, avg_delay))
```



```
# more typical usage of geom_bar where only a single data feature is passed
# plots the number of flights per carrier, since each row is a flight
ggplot(flights) + geom_bar(aes(carrier))
```



4 Boxplots (geom_boxplot())

There is nothing significant for me to add on to the boxplots example, apart from reiterating the interpretation of the boxplot - the thick middle line is the median value, and the ends of the boxes are the 75th and 25th percentile.

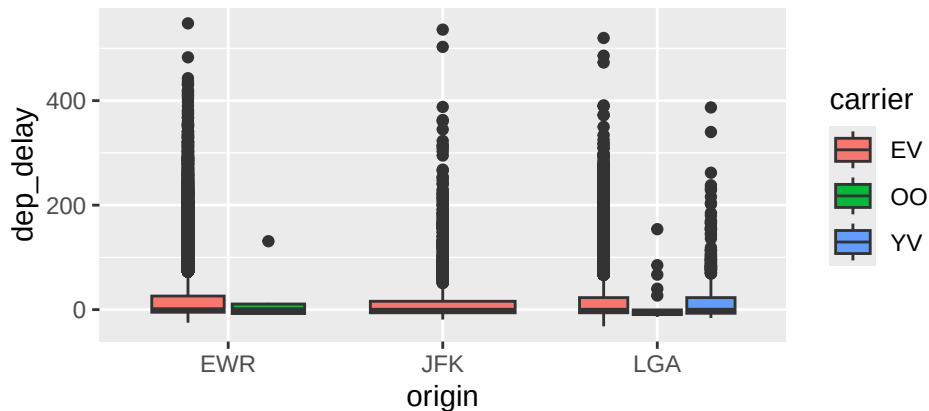
4.1 Zooming in on a plot (coord_cartesian())

However, to zoom in on the plot, `ylim()` used in the tutorial **removes data from the dataset** that are outside of the bounds you specify, *only for the plot*. This is evident when you look at a true zoomed-in version of the data using `coord_cartesian()`.

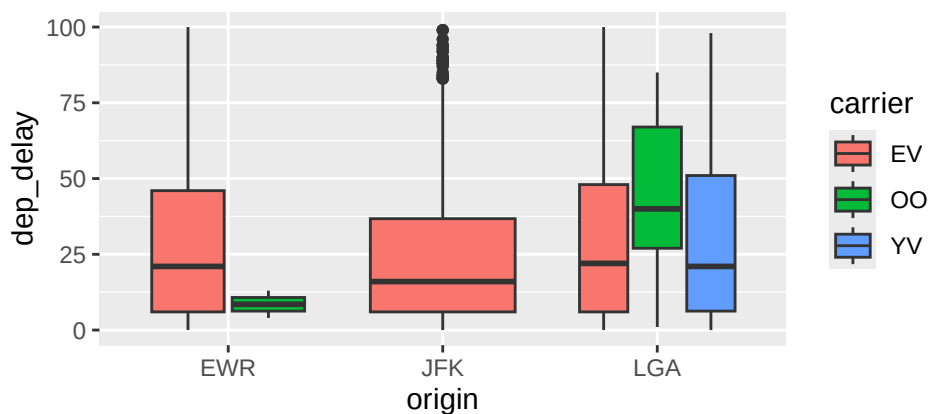
You can remove data from your dataset, but you must be aware that you are removing data, and the reasons you are removing that data!

```
g <- flights %>% filter(carrier %in% c("EV", "OO", "YV"), !is.na(dep_delay))
  ↳ %>%
  ggplot(aes(x=origin, y=dep_delay, fill = carrier)) + geom_boxplot()
g + labs(title="Original plot")
g + ylim(0, 100) + labs(title="ylim()")
g + coord_cartesian(ylim=c(0,100)) + labs(title="coord_cartesian()")
```

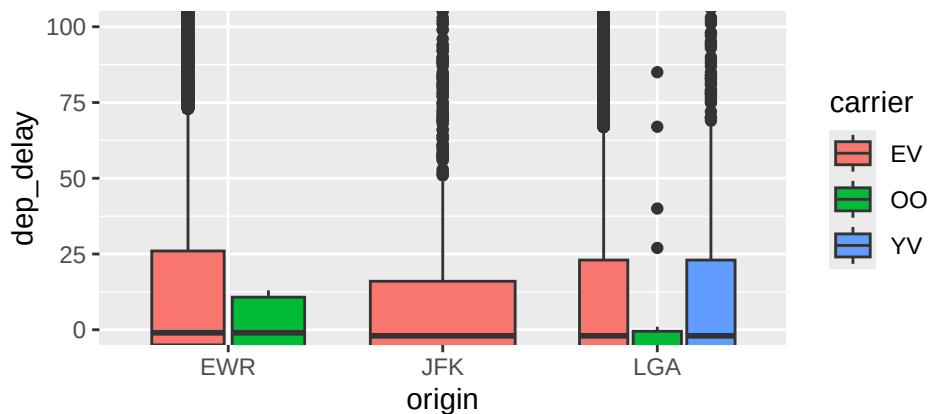
Original plot



ylim()



coord_cartesian()



5 Histograms (`geom_histogram()`) and density plots (`geom_density()`)

Histograms and density plots are closely related, but not exactly the same. They both are meant to show the **distribution** of a *continuous numerical variable*, but differ in that histograms give a *count*, while density plots give the *relative probability* of a value.

Both of these, ultimately, depend on the data, and the parameters of the histogram or density plot. However, the only parameter we will look at in this course is the `bins` parameter of `geom_histogram()`.

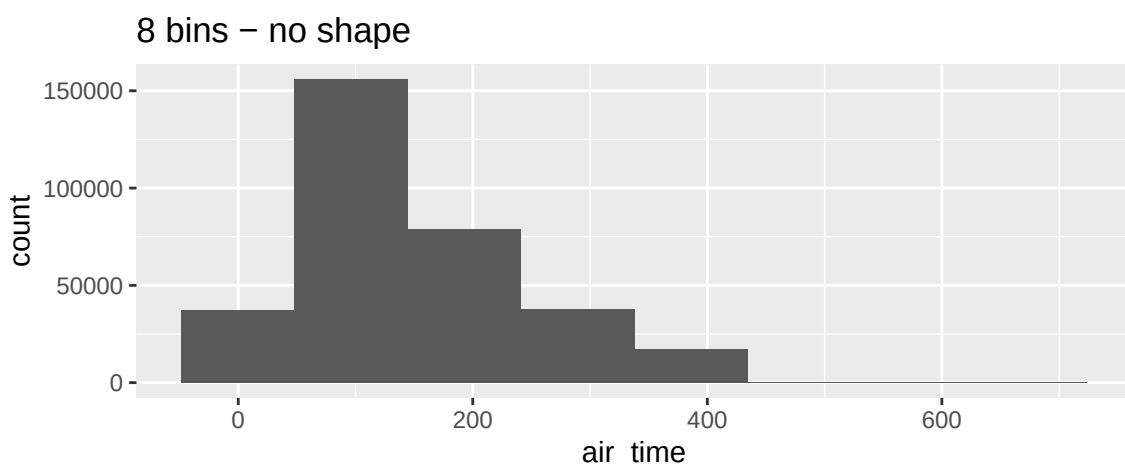
5.1 Histograms (`geom_histogram()`)

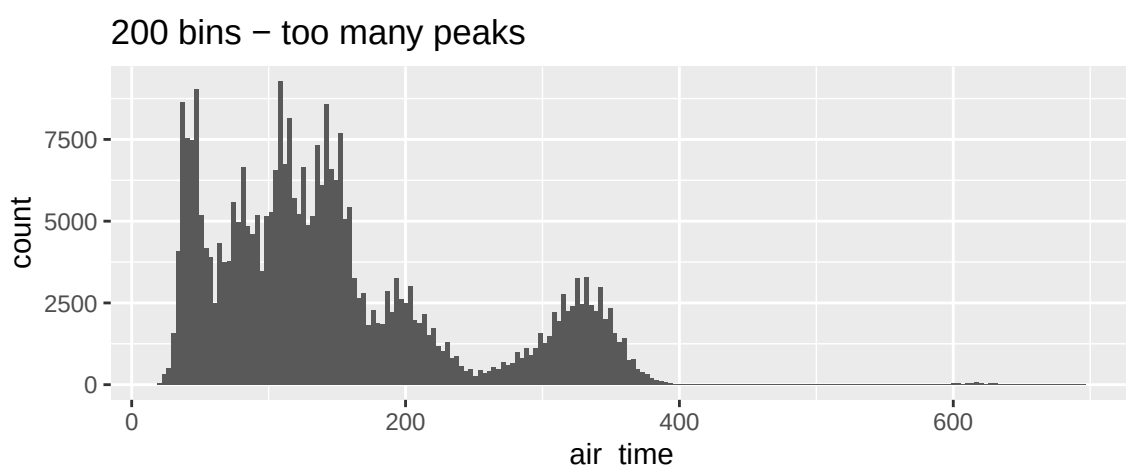
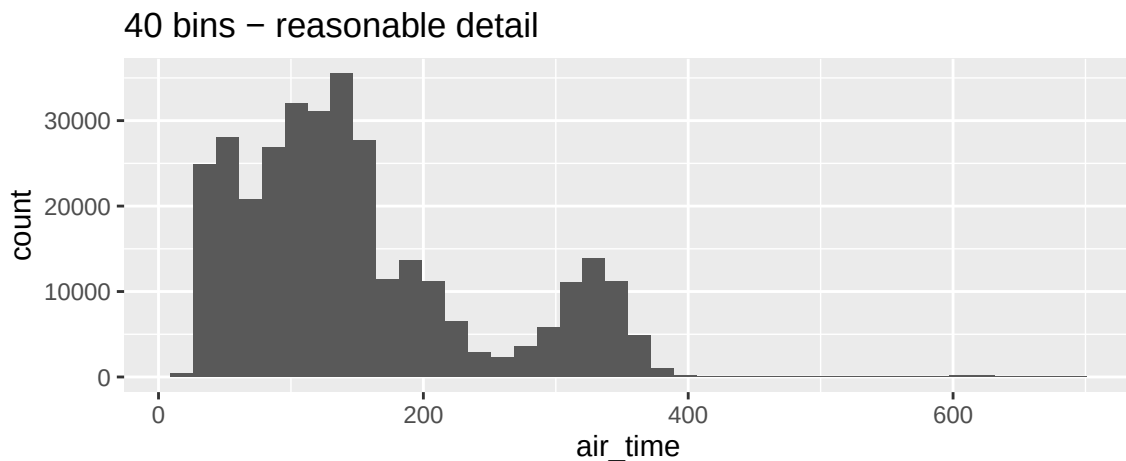
Histograms are plots of the counts of a *single variable*, breaking up the variable in some number of **bins**, each (typically) covering an equally spaced interval. For example, if we are plotting a histogram of students' numerical grades (from 0 to 100), we may choose to split this range up into 10 bins of 10 marks each, or 20 bins of 5 marks each.

In the tutorial sheet, they plot the histogram of the air times of flights. By default, `ggplot()` will use 30 bins, which you can change by altering the `bins` parameter. The choice of the number of bins is quite discretionary - there is no science or correct answer, apart from the general advice of 'not too few, not too many'. Too few bins, and you cannot see the detail of the distribution, too many bins, and you cannot see the general picture of the distribution.

Using 40 bins, you can see that there are three general areas of interest, indicated by the peaks. 0 to 250, 250 to 400, and a small set of values around 600. With 8 bins, the detail of the two peaks is lost, misrepresenting the data. With 200 bins, there are too many peaks which distract from the picture of the distribution.

```
g <- flights %>% filter(!is.na(air_time)) %>% ggplot(aes(air_time))
g + geom_histogram(bins=8) + labs(title="8 bins - no shape")
g + geom_histogram(bins=40) + labs(title="40 bins - reasonable detail")
g + geom_histogram(bins=200) + labs(title="200 bins - too many peaks")
```



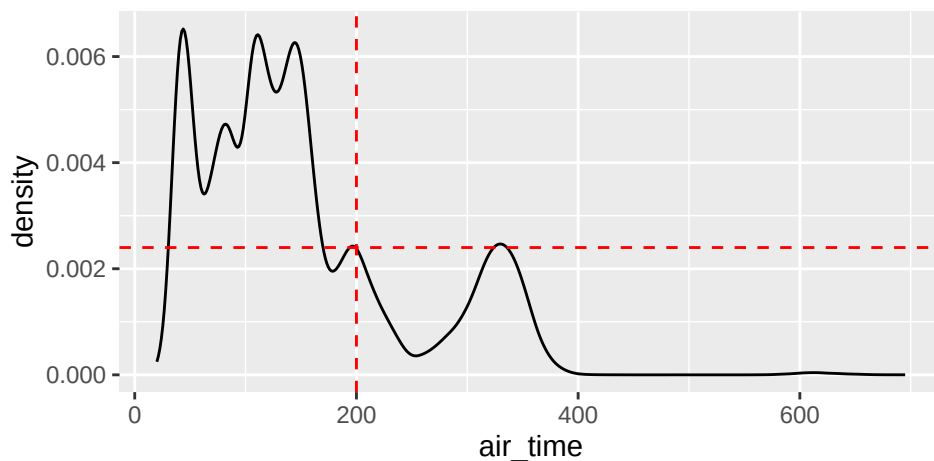


5.2 Density plots (`geom_density()`)

While histograms show the counts of a variable in each bin, density plots show the relative probability of a value occurring. These probabilities are **based on the data** and **the choice of parameters for the density plot**. However, we will not learn about the different parameters the density plot can take.

You can see that the y-axis has been replaced with a density, which is the probability of any value occurring. For example, an `air_time` value of 200 has approximately a $0.0024 = 0.24\%$ chance of occurring based on the density plot. These values are represented by the dashed red lines in the plot below. Note that this probability estimation is based on the dataset we have.

```
ggplot(flights, aes(air_time)) + geom_density(na.rm=T) +  
  geom_vline(xintercept=200, col="red", linetype="dashed") +  
  geom_hline(yintercept=0.0024, col="red", linetype="dashed")
```



6 Matrix scatter plots

The tutorial sheet introduces another plot that *does not use* `ggplot2`, the matrix scatter plot.

6.1 The `iris` dataset

The `iris` dataset is used for these examples, and is built-in to R. You can access it with `iris`. It is a dataset of 150 observations, which consist of 3 species of iris flowers: *setosa*, *versicolor*, and *virginica*. Petal and sepal measurements of 50 of each species are recorded in this dataset.

A quick study of the *structure* of the `iris` dataset shows 4 numerical variables and 1 factor variable (`Species`), which is a categorical variable. `str(iris)` indicates that the factor variable has 3 levels, i.e. 3 unique values.

```
str(iris)
```

```
## 'data.frame':   150 obs. of  5 variables:
##  $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
##  $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
##  $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
##  $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
##  $ Species      : Factor w/ 3 levels "setosa","versicolor",...: 1 1 1 1 1 1 1 1 1 1
```

The summary of the data also gives us some basic statistics on each variable, including the count of each species.

```
summary(iris)
```

```
##      Sepal.Length      Sepal.Width      Petal.Length      Petal.Width
```



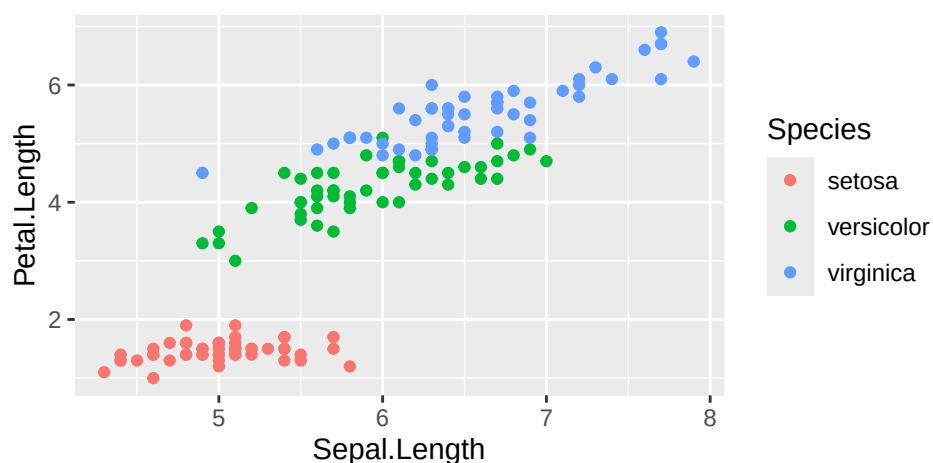
```
##   Min.    :4.300   Min.    :2.000   Min.    :1.000   Min.    :0.100
##   1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.300
##   Median :5.800   Median :3.000   Median :4.350   Median :1.300
##   Mean   :5.843   Mean   :3.057   Mean   :3.758   Mean   :1.199
##   3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100   3rd Qu.:1.800
##   Max.    :7.900   Max.    :4.400   Max.    :6.900   Max.    :2.500
##           Species
##   setosa   :50
##   versicolor:50
##   virginica :50
##
##
##
```

6.2 Scatter plots for clustering (geom_point() with color)

The tutorial uses `geom_point()` to visualise clusters by each species of iris, specifying the `color` parameter to provide a colourful way to visualise the data.

You can see that the species are differentiated by colour, and that there is a clustering pattern among each species.

```
ggplot(iris, aes(x=Sepal.Length, y=Petal.Length, color=Species)) +
  geom_point()
```



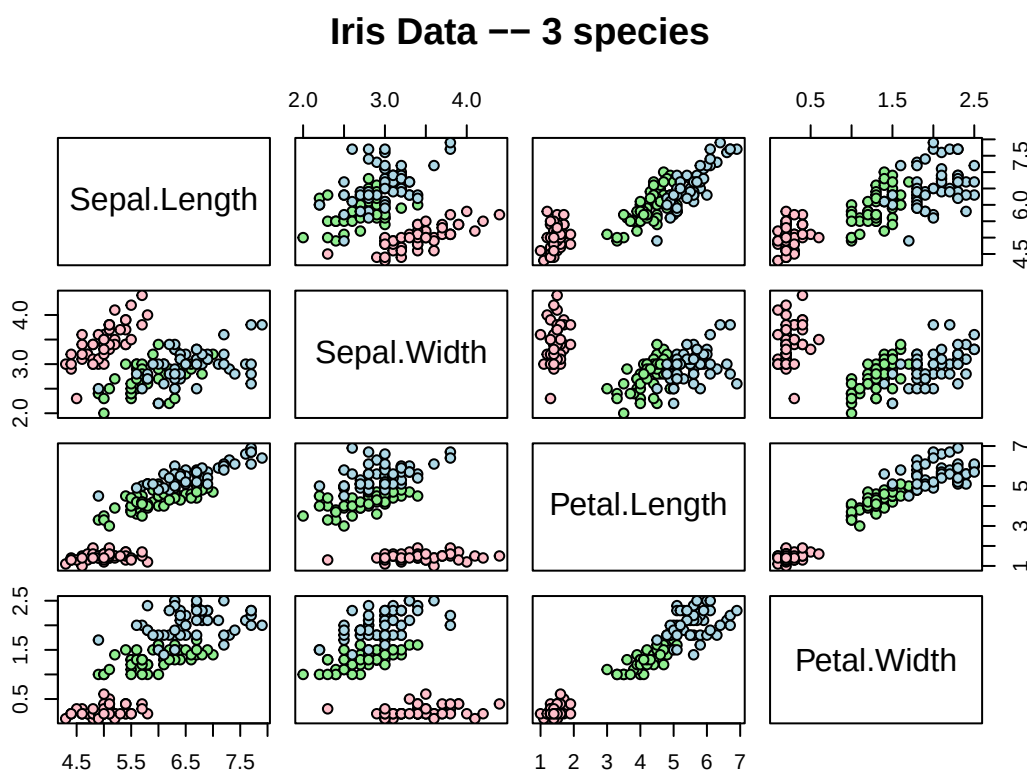
6.3 Matrix scatter plots with the pairs() function

Matrix scatter plots are useful for visualising the relationship between several variables in the same plot. The first method used in the tutorial utilises the built-in `pairs()` function, with `pch` and `bg` parameters.

`pch` refers to the *point character*, i.e. the shape of the character plotted at each point. You can see all the possible point characters with `?pch`. `pch=21` will plot each point as a filled circle.

`bg` refers to the background colour of the point characters, which is equivalent to the `fill` parameter in `ggplot2`. `unclass(iris$Species)` converts the `Species` vector into a vector of 1, 2, or 3, depending on the value. It then indexes from the colour vector, so that each 1 becomes "pink", 2 becomes "light green", and 3 becomes "light blue". These colours are then assigned for each observation in `iris[1:4]`.

```
pairs(iris[1:4], main = "Iris Data -- 3 species",
      pch = 21,
      bg = c("pink", "light green", "light blue")[unclass(iris$Species)])
```



The interpretation of this plot is simpler than it looks. Each feature is plotted against every other feature. For example, looking at the cell labelled `Sepal.Length`, all cells to the *right* of it have `Sepal.Length` as the y-axis, and all cells *below* it have `Sepal.Length` as the x-axis. The colours, added by the `bg` parameter, help us identify clustering patterns in each plot by the species.

6.4 Matrix scatter plots with the `psych` package (`pairs.panels()`)

In the above plots, each variable was plotted against every other variable *twice*, once as the x-axis, and once on the y-axis. The `pairs.panels()` function in the `psych` package (which must be installed)

removes these duplicated graphs, and instead uses the space to display the correlation coefficient of each variable against every other variable.

We use the Pearson correlation coefficient, which is **a measure to quantify the linear relationship between two quantitative variables**. It varies between -1 and 1. A positive (negative) correlation coefficient indicates an upward (downward) sloping line of best fit. Values closer to -1 or 1 imply that the data is closer to the line. For some visual representation of different correlation values, you may look at the Wikipedia page on Pearson's correlation coefficient.

There are many settings that can be changed, but I personally do not think that this is a function that you need to memorise its use deeply for this course. Focus instead on other key concepts, especially in the coming weeks.

```
library(psych)
pairs.panels(iris[1:4],           # dataset is the first four columns of iris
  method = "pearson",           # use the Pearson correlation coefficient
  hist.col = "steelblue",       # set the histogram column colour
  pch = 21, bg = c("pink", "light green", "light blue")[iris$Species],
  density = TRUE,               # show density plots along the diagonal
  ellipses = FALSE              # hide correlation ellipses
)
```

